Звіт про виконання завдання № 5.2

Сума ряду Тейлора за допомогою функцій

Варіант 7

Виконав: Джамалов Олександр Олегович, група РІ-11

1. Мета роботи

Навчитися реалізовувати циклічні алгоритми з рекурентними співвідношеннями у вигляді окремих функцій та передавати їм усю інформацію через параметри.

2. Умова завдання

Обчислити і вивести на екран у вигляді таблиці значення функції, заданої за допомогою ряду Тейлора, на інтервалі від $x_{поч}$ до $x_{кін}$ з кроком dx та точністю ε. Параметри $x_{поч}$, $x_{кін}$, dx, eps вводяться з клавіатури. Таблиця має містити заголовок та шапку. Кожний рядок таблиці має містити значення аргументу, значення функції, значення суми ряду та кількість порахованих доданків. Суму ряду Тейлора та значення доданку обчислювати за допомогою допоміжних алгоритмів, реалізованих за допомогою окремих функцій. При обчисленні значення доданків використовувати рекурентні співвідношення. Всю необхідну функціям інформацію слід передавати лише за допомогою параметрів.

Варіант 7.

$$\operatorname{arcctg} x = \frac{\pi}{2} + \sum_{n=0}^{\infty} \frac{(-1)^{n+1} x^{2n+1}}{2n+1} = \frac{\pi}{2} - x + \frac{x^3}{3} - \frac{x^5}{5} - \ldots, \quad |x| <= 1.$$

Рисунок 1 – Умова завдання 5.2, варіант 7

3. Математична модель

Доданки ряду: $a_n = (-1)^{n+1} x^{2n+1} / (2n + 1)$, $n = 0, 1, 2, \ldots$ Перший доданок $a_0 = -x$.

Відношення сусідніх доданків: $a_n / a_{n-1} = -x^2 (2n - 1) / (2n + 1)$, звідки рекурентна формула $a_n = a_{n-1} \cdot (-x^2 (2n - 1) / (2n + 1))$, $n \geq 1$.

$S(x) = \pi/2 + \Sigma a_n$. Сумування припиняється, коли $|a_n| < \varepsilon$. Для $|x| \leq 1$ ряд знакозмінний і модулі доданків спадають, тому похибка не перевищує ε. Для контролю виводиться точне значення arcctg x = π/2 − arctg x. Кількість доданків включає a_0, але не константу π/2.

Обмеження: $-1 \leq x_{поч} \leq x_{кін} \leq 1$, $dx > 0$, $\varepsilon > 0$, кількість рядків – не більше 10^6, кількість доданків – не більше 10^7.

4. Структура програми

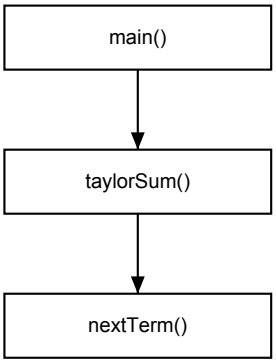


Рисунок 2 – Структурна схема програми (ієрархія викликів)

Таблиця 1 – Специфікація функцій програми

Функція	Призначення	Вхідні параметри	Результат	Викликає
int main()	введення даних, побудова таблиці значень	–	код завершення	taylorSum
double taylorSum(double x, double eps, int& count)	сума ряду Тейлора з точністю eps	x – аргумент; eps – точність	π/2 + сума; count – кількість доданків (параметр-посилання)	nextTerm
double nextTerm(double x, int i, double a)	наступний доданок за рекурентною формулою	x; i – номер; a – попередній доданок	доданок a_i	–

Таблиця 2 – Змінні та константи програми

Ім'я	Тип	Функція	Призначення
xp, xk	double	main	початок і кінець інтервалу
dx	double	main	крок зміни x
eps	double	main	точність обчислення суми
rows	int	main	номер останнього рядка таблиці
i	int	main	номер поточного рядка
x	double	main	поточний аргумент
sum	double	main / taylorSum	сума ряду
exact	double	main	точне значення π/2 − arctg x
count	int	main / taylorSum	кількість врахованих доданків
a	double	taylorSum / nextTerm	поточний доданок
MAX_TERMS	const int	глобальна	10^7 – межа кількості доданків
MAX_ROWS	const double	глобальна	10^6 – межа кількості рядків

5. Алгоритми програми

Блок-схеми виконано за ГОСТ 19.701-90 (ДСТУ ISO 5807:2016): «Початок» і «Кінець» – символ «термінатор», введення/виведення – «дані» (паралелограм), обчислення – «процес», виклик функції – «зумовлений процес», умова – «рішення». Лінії потоку, що йдуть зверху вниз і зліва направо, стрілками не позначено; стрілку показано лише на лінії повернення (повторення циклу).

UML-діаграми діяльності (UML 2.5) виконано в рамці «act»: початковий вузол – зафарбоване коло, кінцевий – коло з точкою, дії – прямокутники із заокругленими кутами (дія-виклик позначена символом «вила»), розгалуження та злиття – ромби, умови переходів – сторожові вирази у квадратних дужках. Усі переходи позначено стрілками.

5.1. Головна функція main()

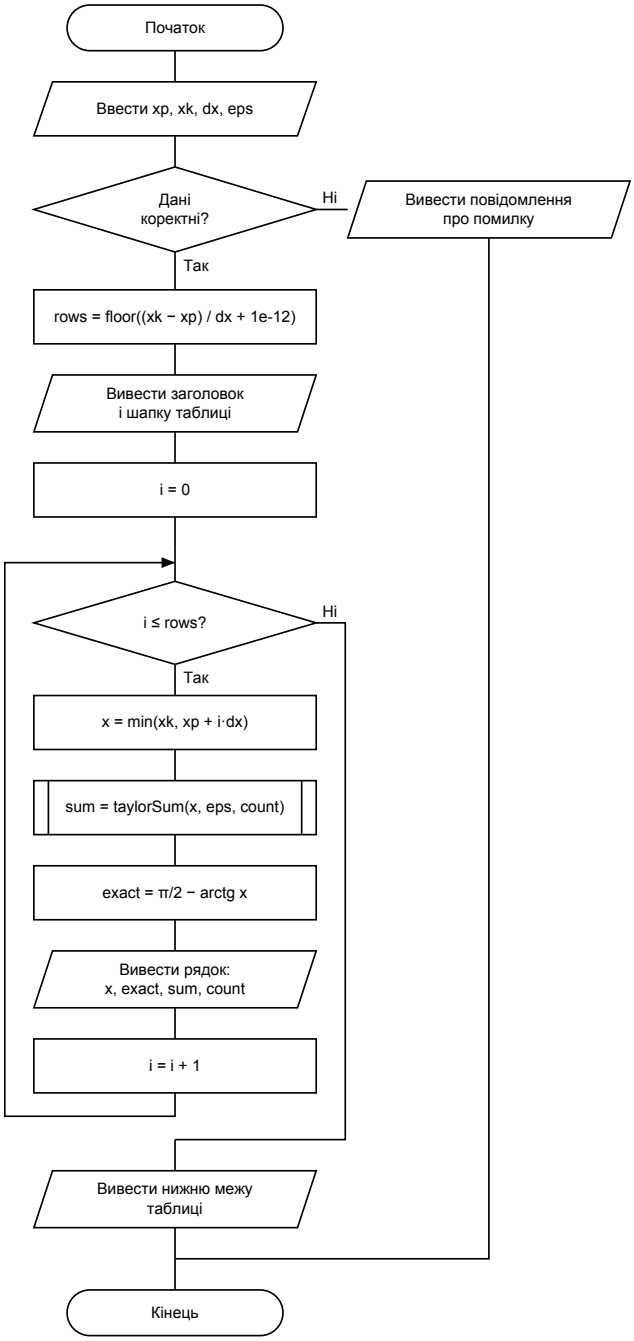


Рисунок 3 – Блок-схема функції main()

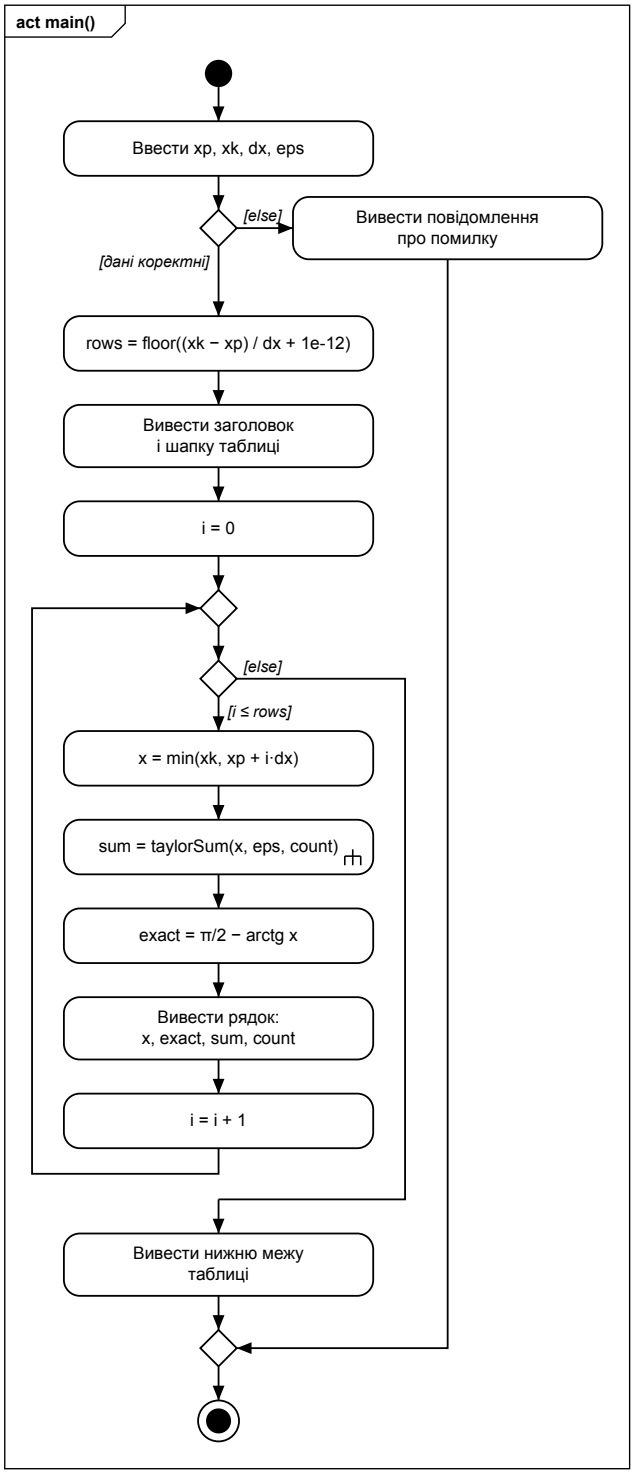


Рисунок 4 – UML-діаграма діяльності функції main()

5.2. Функція taylorSum(x, eps, count)

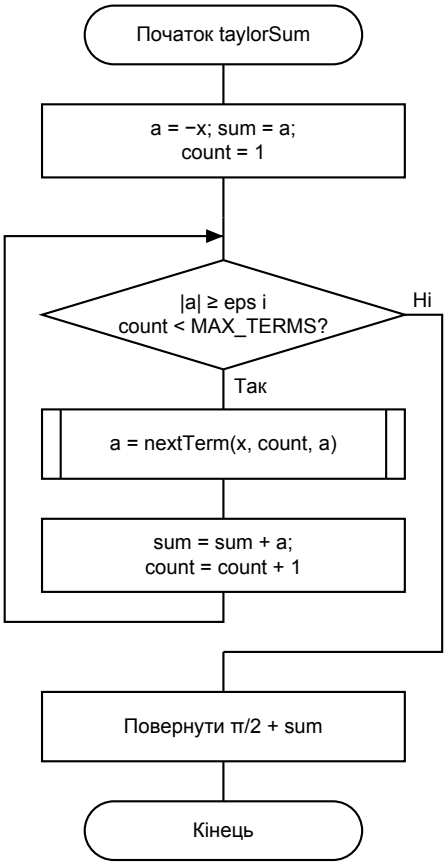


Рисунок 5 – Блок-схема функції taylorSum(x, eps, count)

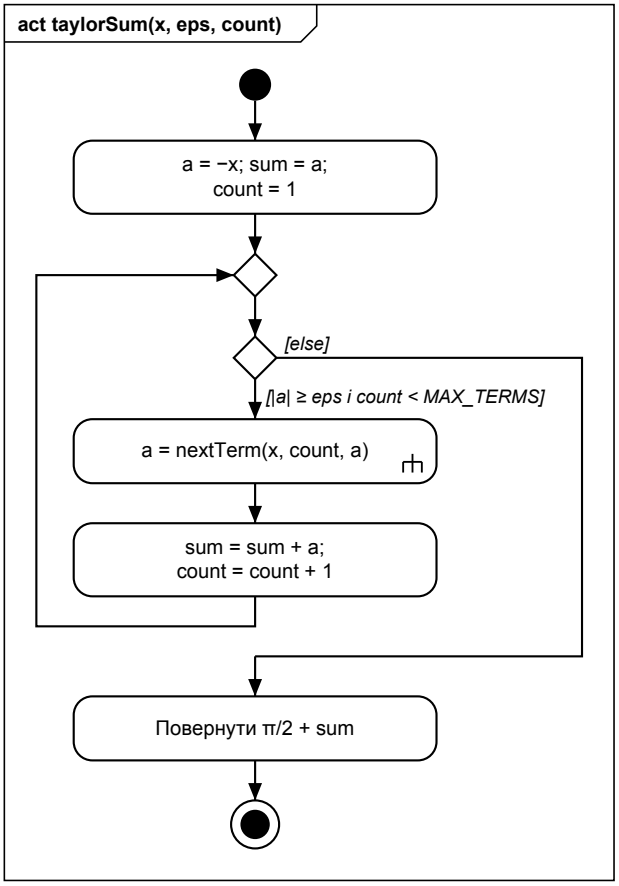


Рисунок 6 – UML-діаграма діяльності функції taylorSum(x, eps, count)

5.3. Функція nextTerm(x, i, a)

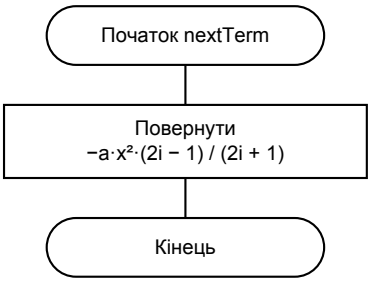


Рисунок 7 – Блок-схема функції nextTerm(x, i, a)

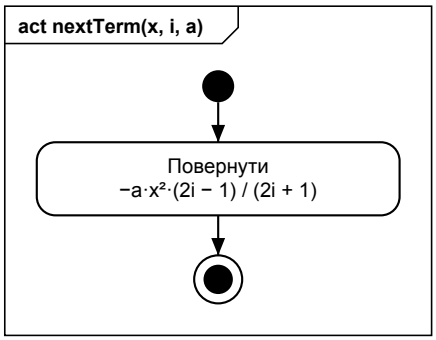


Рисунок 8 – UML-діаграма діяльності функції nextTerm(x, i, a)

6. Текст програми

Кожна дія програми супроводжується коментарем. Головну функцію main() вимкнено директивою #ifndef UNIT_TEST, щоб файл можна було підключити до тестів.

Файл `lab_5_2.cpp`

```
// Лабораторна робота 5.2, варіант 7. Джамалов О. О., PI-11.
// arcctg(x) = pi/2 + sum_{n>=0} (-1)^(n+1) x^(2n+1) / (2n+1),  |x| <= 1.
#include <algorithm> // min
#include <cmath>     // abs, acos, atan, floor, isfinite
#include <iomanip>   // setw, setprecision, fixed
#include <iostream>  // cin, cout, cerr
using namespace std;

const int MAX_TERMS = 10000000;  // захист від нескінченного циклу
const double MAX_ROWS = 1000000; // найбільша кількість рядків таблиці

// Доданок за рекурентною формулою:
// a_i = a_(i-1) * (-x^2 * (2i-1) / (2i+1)), i >= 1.
double nextTerm(double x, int i, double a)
{
    return -a * x * x * (2.0 * i - 1) / (2.0 * i + 1); // новий доданок
}

// Сума ряду; count - кількість доданків (разом з a_0 = -x).
double taylorSum(double x, double eps, int& count)
{
    double a = -x;                          // перший доданок a_0
    double sum = a;                         // сума починається з a_0
    count = 1;                              // враховано один доданок
    while (abs(a) >= eps && count < MAX_TERMS) // поки доданок не менший за eps
    {
        a = nextTerm(x, count, a);          // наступний доданок
        sum += a;                           // додаємо його до суми
        ++count;                            // збільшуємо лічильник
    }
    return acos(-1.0) / 2 + sum;            // pi/2 + сума ряду
}

#ifndef UNIT_TEST
int main()
{
    double xp, xk, dx, eps;                 // межі, крок і точність
    cout << "x_begin x_end dx eps: ";       // запрошення до введення
    if (!(cin >> xp >> xk >> dx >> eps) ||  // перевірка введення
        !isfinite(xp) || !isfinite(xk) || !isfinite(dx) || !isfinite(eps) ||
        xp < -1 || xk > 1 || xp > xk || dx <= 0 || eps <= 0 ||
        (xk - xp) / dx > MAX_ROWS)
    {
        cerr << "Error: -1 <= x_begin <= x_end <= 1, dx > 0, eps > 0.\n"; // помилка
        return 1;                           // завершення з кодом помилки
    }

    const int rows = static_cast<int>(floor((xk - xp) / dx + 1e-12)); // номер останнього рядка
    const char* line =                      // межа таблиці
        "+-------------+----------------+----------------+----------+\n";
    cout << "Lab 5.2 / Variant 7\n" << line; // заголовок і верхня межа
    cout << "|      x      |    arcctg(x)    |       sum       |  terms   |\n"
         << line;                           // шапка таблиці
    cout << fixed << setprecision(8);       // формат чисел
    for (int i = 0; i <= rows; ++i)         // перебір точок інтервалу
    {
        const double x = min(xk, xp + i * dx);        // поточний аргумент
        int count;                                    // кількість доданків
        const double sum = taylorSum(x, eps, count);  // сума ряду
        const double exact = acos(-1.0) / 2 - atan(x); // точне значення
        cout << "| " << setw(11) << x << " | " << setw(15) << exact
             << " | " << setw(15) << sum << " | " << setw(8) << count
             << " |\n";                               // рядок таблиці
    }
    cout << line;                           // нижня межа таблиці
    return 0;                               // успішне завершення
}
#endif
```

7. Автоматизоване тестування

Файл `test_5_2.cpp`

```
// Unit-тести функцій taylorSum і nextTerm із завдання 5.2.
#define UNIT_TEST       // вимикаємо main() у файлі програми
#include "lab_5_2.cpp"  // підключаємо функції

int main()
{
    int failed = 0;                                  // лічильник невдалих тестів
    const double xs[] = {-1, -0.5, 0, 0.5, 1};       // тестові аргументи
    for (double x : xs)                              // перебір аргументів
    {
        const double eps = abs(x) == 1 ? 1e-5 : 1e-10; // точність для x
        int count;                                   // кількість доданків
        const double actual = taylorSum(x, eps, count);       // сума ряду
        const double expected = acos(-1.0) / 2 - atan(x);     // еталон arcctg(x)
        const bool ok = abs(actual - expected) < eps + 1e-14  // перевірка точності
                        && count >= 1 && (x != 0 || count == 1);
        cout << (ok ? "PASS" : "FAIL") << " x=" << x
             << " terms=" << count << '\n';          // результат тесту
        if (!ok) ++failed;                           // рахуємо помилку
    }
    const bool termOk = abs(nextTerm(0.5, 1, -0.5)   // a_1 для x = 0.5
                            - 1.0 / 24) < 1e-15;     // має дорівнювати 1/24
    cout << (termOk ? "PASS" : "FAIL") << " nextTerm\n"; // результат тесту
    if (!termOk) ++failed;                           // рахуємо помилку
    cout << "5.2: " << 6 - failed << "/6 passed\n";  // підсумок
    return failed ? 1 : 0;                           // код завершення для CTest
}
```

Фактичний вивід тестової програми:

```
PASS x=-1 terms=50001
PASS x=-0.5 terms=15
PASS x=0 terms=1
PASS x=0.5 terms=15
PASS x=1 terms=50001
PASS nextTerm
5.2: 6/6 passed
```

Таблиця 3 – Результати unit-тестів

№	Перевірка	x	eps	Еталон	Результат
1	taylorSum	−1	10^{-5}	π/2 − arctg x	пройдено
2	taylorSum	−0,5	10^{-10}	π/2 − arctg x	пройдено
3	taylorSum	0	10^{-10}	π/2, 1 доданок	пройдено
4	taylorSum	0,5	10^{-10}	π/2 − arctg x	пройдено
5	taylorSum	1	10^{-5}	π/2 − arctg x	пройдено
6	nextTerm(0,5; 1; −0,5)	0,5	–	1/24	пройдено

8. Результати виконання програми

Компілятор clang++, стандарт C++17, ключі -Wall -Wextra -Wpedantic (попереджень немає). Вхідні дані: `-1 1 0.5 0.00001`.

```
x_begin x_end dx eps: -1 1 0.5 0.00001
Lab 5.2 / Variant 7
+-------------+-----------------+-----------------+----------+
|      x      |    arcctg(x)    |       sum       |  terms   |
+-------------+-----------------+-----------------+----------+
| -1.00000000 |      2.35619449 |      2.35619949 |    50001 |
| -0.50000000 |      2.03444394 |      2.03444560 |        7 |
|  0.00000000 |      1.57079633 |      1.57079633 |        1 |
|  0.50000000 |      1.10714872 |      1.10714705 |        7 |
|  1.00000000 |      0.78539816 |      0.78539316 |    50001 |
+-------------+-----------------+-----------------+----------+
```

9. Посилання на Git-репозиторій

https://github.com/dzhamsasha55-dev/lab-5

Висновки

Суму ряду Тейлора та доданок реалізовано окремими функціями; уся інформація передається через параметри, кількість доданків повертається через параметр-посилання. Рекурентна формула дає змогу обчислювати доданки без піднесення до степеня.

Результати виводяться у вигляді таблиці з рамкою. На кінцях інтервалу |x| = 1 ряд збігається повільно, тому кількість доданків значно більша. Усі 6 unit-тестів пройдено.